

32. LRU Page Replacement

Aim

To write and execute a C program to simulate the **Least Recently Used (LRU) page replacement technique** of memory management and calculate the total number of page faults.

Algorithm

1. Start the program.
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of available frames.
5. Initialize all frames as empty.
6. Read each page from the reference string one by one.
7. Check whether the page is already present in any frame.
8. If the page is present, it is a **page hit** and update its recent-use information.
9. If the page is not present, it is a **page fault**.
10. If an empty frame is available, place the new page in that frame.
11. If all frames are occupied, find the page that has been **least recently used**.
12. Replace the least recently used page with the new page.
13. Increment the page-fault count.
14. Display the contents of the frames after each page reference.
15. Repeat the process until all pages are processed.
16. Display the total number of page faults.
17. Stop the program.

Code:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10], recent[10];
    int n, f, i, j, k, pos, fault = 0, found, min;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
    printf("Enter number of frames: ");
    scanf("%d", &f);
    for (i = 0; i < f; i++) {
```

```

    frames[i] = -1;
    recent[i] = 0;
}
for (i = 0; i < n; i++) {
    found = 0;
    for (j = 0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            recent[j] = i;
            break;
        }
    }
    if (!found) {
        fault++;
    }
    pos = -1;
    for (j = 0; j < f; j++) {
        if (frames[j] == -1) {
            pos = j;
            break;
        }
    }
    if (pos == -1) {
        min = recent[0];
        pos = 0;
        for (j = 1; j < f; j++) {
            if (recent[j] < min) {
                min = recent[j];
                pos = j;
            }
        }
    }
}

```

```

    }

    frames[pos] = pages[i];
    recent[pos] = i;
}

printf("Page %d: ", pages[i]);
for (j = 0; j < f; j++)
    printf("%d ", frames[j]);
printf("\n");
}

printf("\nTotal Page Faults = %d\n", fault);

return 0;
}

```

Output:

main.c	Run	Output
<pre> 3 int main() { 23 for (i = 0; i < n; i++) { 34 if (!found) { 43 } 44 45 if (pos == -1) { 46 min = recent[0]; 47 pos = 0; 48 49 for (j = 1; j < f; j++) { 50 if (recent[j] < min) { 51 min = recent[j]; 52 pos = j; 53 } 54 } 55 } 56 57 frames[pos] = pages[i]; 58 recent[pos] = i; 59 } 60 61 printf("Page %d: ", pages[i]); 62 for (j = 0; j < f; j++) 63 printf("%d ", frames[j]); 64 printf("\n"); </pre>		<pre> Enter number of pages: 12 Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5 Enter number of frames: 3 Page 1: 1 -1 -1 Page 2: 1 2 -1 Page 3: 1 2 3 Page 4: 4 2 3 Page 1: 4 1 3 Page 2: 4 1 2 Page 5: 5 1 2 Page 1: 5 1 2 Page 2: 5 1 2 Page 3: 3 1 2 Page 4: 3 4 2 Page 5: 3 4 5 Total Page Faults = 10 === Code Execution Successful === </pre>